Introduction to logic (maths)

Jessica Taberner

Summary

Propositional logic is key to understanding and building rigorous arguments. This guide uses mathematical language to outline what propositional logic is, what logical connectives are, and how to use truth tables.

What is propositional logic?

Propositional logic forms a foundation for clear and rigorous thinking. It lets you communicate ideas clearly, analyze arguments, and detect errors in reasoning. Having a good understanding of propositional logic can be very useful when working on mathematical proofs, but it's also crucial in computer science and AI, where logical rules are needed to develop algorithms or program decision-making. Understanding propositional logic can help you develop your problem solving and reasoning ability, skills that will prove useful in real world decision-making.

i Definition of propositional logic

Propositional logic studies statements that are either true or false. You call these statements **propositions**.

Propositional logic lets you analyze the truth value of statements. In order to properly assess the truth value of a statement, it should be a proposition, such as '$5 \leq 7$.' For example, $x \leq y$, is not a proposition; this is an **open sentence** as the variables are not specified. This will be true for some values of x and y, and false for others.

i **Example 1**

"$2 \cdot 3 = 6$," is one example of a proposition. This is always true.
"$2 \cdot 3 = 5$." Again, this is a proposition. This is always false.
"n is even." This is an **open sentence**, so it cannot be a proposition. It could be true or false depending on the value of n.
"$x^2 = 2$." Again, this is an **open sentence**, so it cannot be a proposition. It could be true or false depending on the value of x.

While statements such as $x = 1$ are not propositions, within propositional logic you can often treat them as such. Throughout this guide you can take P, Q, and R as placeholders for propositions or open sentences. You can use the capital letters here to distinguish them from variables.

Connectives

In order to analyze more complex statements, you'll need connectives. Connectives allow you to combine sentences with a variety of different implications. You'll now see how to define a few key connectives. For these definitions you can take P and Q to be two statements, either propositions or open sentences.

Definition of 'and'

If P and Q are statements, using **conjunction** you can form the statement 'P **and** Q,' often written $P \wedge Q$. The truth value of $P \wedge Q$ is dependent on the truth values for P and Q. The statement, $P \wedge Q$, takes the value true if both P and Q are true, and false otherwise.

One example of this might be "n is even and $n < 6$." This statement is only true if both parts of the statement are true, and is false otherwise.

Definition of 'or'

If P and Q are statements, using **disjunction** you can form the statement 'P **or** Q,' often written $P \vee Q$. The truth value of $P \vee Q$ is dependent on the truth values for P and Q. The statement, $P \vee Q$, takes the value true if at least one of P and Q are true, and false otherwise.

An example of this would be '$x > 2$ or $x < 2$.' This statement is true if either element is true, it's only false if neither part is true and $x = 2$.

Example 2

Let P stand for "x is odd" and Q stand for "$x > 0$."
Now, using connectives, you can consider the statements $P \wedge Q$ and $P \vee Q$.
The first statement, $P \wedge Q$, will mean "x is odd and $x > 0$." This will be true if x is both odd and positive, and it will be false when x is odd and negative, even and positive, or even and negative.
The second statement, $P \vee Q$, will mean "x is odd or $x > 0$." This will be true if x is odd and positive, odd and negative, or even and positive. It will only be false when x is even and negative.

Definition of 'not'

If P is a statement, using **negation** you can form the statement '**not** P,' often written $\neg P$. The truth value of $\neg P$ is dependent on the truth value of P. The statement, $\neg P$, is true if P is false, and $\neg P$ is false if P is true.

One example of this might be "x is a not multiple of 5." This is only false if it's inverse is true and x is a multiple of 5.

Tip

You should note that when you negate an inequality, not only will you flip the sign, you'll also change the 'strictness' of the inequality.
For example, the negation of $x > 0$ is $x \leq 0$. Similarly, the negation of $x \geq 0$ is $x < 0$.

Example 3

Let P stand for "$x > 5$" and Q stand for "$y < 5$."
Let's consider the statement $\neg(P \vee Q)$.
$P \vee Q$ will mean "$x > 5$" **or** $y < 5$," but how will the negation change this statement?
You want to negate both P and Q, as well as the connective, **or**.
Negating the **or** connective will result in **and**, similarly negating **and** will result in **or**.
Here $\neg P$ will mean "$x \leq 5$", and $\neg Q$ will mean "$y \geq 5$".
So, $\neg(P \vee Q)$ will mean "$x \leq 5$ **and** $y \geq 5$."

Definition of 'implies'

If P and Q are statements, using **implication** you can form the statement 'P **implies** Q,' often written $P \to Q$. The truth value of $P \to Q$ is dependent on the truth values for P and Q. The statement, $P \to Q$, is false if P is true and Q is false, and is true in

all other cases. You should note that **order matters** for this connective.

An example of an implication might be, "If x is even, then y is odd." If x is not even the statement is true if y is odd or not, the only way it could be false is if x is even and so is y.

i Definition of 'if and only if'

If P and Q are statements, using **equivalence** you can form the statement 'P **if and only if** Q,' often written $P \leftrightarrow Q$. The truth value of $P \leftrightarrow Q$ is dependent on the truth values for P and Q. The statement, $P \leftrightarrow Q$, is true if both P and Q are true, or if both P and Q are false, and it's false if P and Q take different values.

One example of this is, "x is even if and only if x is a multiple of 2." This is true when both parts are true, or both aren't.

i **Example 4**

Let's try breaking down a statement and writing it symbolically.
"$x > 0$ if and only if y is odd."
If you let P stand for "$x > 0$," and Q stand for "y is odd," then you can write this statement as an equivalence. This is true when either both parts are true, or when both are false: $P \rightarrow Q$.
If instead your statement was "if $x > 0$ then y is odd," then you would write this statement as an implication. This is only false when $x > 0$ and y is even: $P \leftrightarrow Q$.

Truth tables

These connectives are best understood using truth tables. Truth tables provide a convenient, visual way of compiling all truth values of a statement. Truth tables are very useful for determining when more complex statements are true or false.

i Definition of truth table

A **truth table** is a systematic way of showing all possible truth values of a logical statement. It lists every combination of truth values for the components the statement and shows the resulting truth value of the overall statement.

Example 5

Here are the truth tables for all of the above connectives:

$P \wedge Q$

P	Q	$P \wedge Q$
T	T	T
T	F	F
F	T	F
F	F	F

$P \vee Q$

P	Q	$P \vee Q$
T	T	T
T	F	T
F	T	T
F	F	F

$\neg P$

P	$\neg P$
T	F
F	T

$P \to Q$

P	Q	$P \to Q$
T	T	T
T	F	F
F	T	T
F	F	T

$P \leftrightarrow Q$

Tip

If there are n propositions in your statement, there will be 2^n rows in your table to account for all possible true and false pairings.

Example 6

Let's review the statement, "$x \nless 2$ and $y = 5$."
Let P stand for "$x < 2$," and Q stand for "$y = 5$."
You can then write the statement as:$(\neg P) \wedge Q$.
Let's start the truth table with all possible truth values for P and Q.

P	Q
T	T
T	F
F	T
F	F

Now you want to negate P, so switch all true values to false, and false to true.

P	$\neg P$	Q
T	F	T
T	F	F
F	T	T
F	T	F

Now you can compare the $\neg P$ and Q columns, using what you know about the $\wedge$ connective. You take the value true only when both elements are true, and false otherwise.

P	$\neg P$	Q	$(\neg P) \wedge Q$
T	F	T	F
T	F	F	F
F	T	T	T
F	T	F	F

Example 7

Let's change this statement slightly. "It's not the case that $x < 2$ and $y = 5$"
Let P still stand for "$x < 2$," and Q stand for "$y = 5$."
You can then write the statement as:$\neg(P \wedge Q)$.
Let's start the truth table with all possible truth values for P and Q.

P	Q
T	T
T	F
F	T
F	F

Now you want to add a column for $P \wedge Q$. You take the value true only when both elements are true, and false otherwise.

P	Q	$P \wedge Q$
T	T	T
T	F	F
F	T	F
F	F	F

Now you want to negate $P \wedge Q$. You take the value true only when $P \wedge Q$ is false.

P	Q	$P \wedge Q$	$\neg(P \wedge Q)$
T	T	T	F
T	F	F	T
F	T	F	T
F	F	F	T

Warning

These examples showcase the importance of bracketing in statements. You can notice that in example 6, P and Q being false resulted in $(\neg P) \wedge Q$ being false, but in example 7, P and Q being false resulted in $\neg(P \wedge Q)$ being true.

Example 8

Let P stand for "$x > 0$," let Q stand for "$y > 4$" and let R stand for "$z = 2n$."
Let's review the statement "If x is positive, then $y > 4$ and z is even."
With propositional logic, you can write this as: $P \to (Q \wedge R)$.
Let's compute the truth table for this. Since there are 3 propositions, you'll need $2^3 = 8$ rows.

P	Q	R
T	T	T
T	F	T
T	T	F
T	F	F
F	T	T
F	F	T
F	T	F
F	F	F

Now you want to add a column for $Q \wedge R$. You take the value true only when both elements are true, and false otherwise.

P	Q	R	$Q \wedge R$
T	T	T	T
T	F	T	F
T	T	F	F
T	F	F	F
F	T	T	T
F	F	T	F
F	T	F	F
F	F	F	F

You can try using the truth table builder below to generate truth tables for some more complex propositions.

```
#| '!! shinylive warning !!': |
#|   shinylive does not work in self-contained HTML documents.
#|   Please set `embed-resources: false` in your metadata.
#| standalone: true
#| viewerHeight: 530
library(shiny)
library(bslib)
library(DT)

# Generate truth table
generate_truth_table <- function(vars) {
  expand.grid(rep(list(c(TRUE, FALSE)), length(vars))) |>
    setNames(vars)
}

# Convert logic symbols to R syntax (UPDATED for brackets)
convert_logic <- function(expr) {
  expr <- gsub("¬", "!", expr)
  expr <- gsub(" ", "&", expr)
  expr <- gsub(" ", "|", expr)

  # implication (supports brackets now)
  expr <- gsub("(.*)→(.*)", "(!\\1 | \\2)", expr)

  # biconditional (supports brackets now)
  expr <- gsub("(.*) (.*)", "(\\1 == \\2)", expr)

  expr
}

# Safe evaluation
evaluate_expr <- function(expr, data) {
  expr <- convert_logic(expr)

  tryCatch({
    eval(parse(text = expr), envir = data)
```

```
  }, error = function(e) {
    rep(NA, nrow(data))
  })
}

ui <- fluidPage(

  tags$head(
    tags$style(HTML("
      .T { color:#3f68b6; font-weight:bold; font-size:18px; }
      .F { color:#db4315; font-weight:bold; font-size:18px; }

      table {
        border-collapse: collapse;
        margin-top: 20px;
        font-family: sans-serif;
        min-width: 300px;
      }

      th {
        background-color: #f5f5f5;
        padding: 10px 16px;
        text-align: center;
        font-weight: 600;
        border-bottom: 2px solid #ddd;
      }

      td {
        padding: 10px 16px;
        text-align: center;
        border-bottom: 1px solid #eee;
      }

      tr:nth-child(even) {
        background-color: #fafafa;
      }

      tr:hover {
```

```
      background-color: #f1f7ff;
    }

    button {
      margin: 2px;
    }
  "))
),

titlePanel("Truth Table Builder"),

sidebarLayout(
  sidebarPanel(
    textInput("vars", "Variables:", "p q"),
    textInput("expr", "Expression:", ""),

    h4("Variables"),
    uiOutput("var_buttons"),

    h4("Connectives"),
    div(
      actionButton("not", "¬"),
      actionButton("and", " "),
      actionButton("or", " "),
      actionButton("imp", "→"),
      actionButton("iff", " ")
    ),

    h4("Brackets"),
    div(
      actionButton("lpar", "("),
      actionButton("rpar", ")")
    ),

    br(),
    actionButton("clear", "Clear")
  ),
```

```
    mainPanel(
      htmlOutput("table")
    )
  )
)

server <- function(input, output, session) {

  # Reactive variable list
  vars <- reactive({
    v <- strsplit(input$vars, "\\s+")[[1]]
    v[v != ""]
  })

  # Variable buttons
  output$var_buttons <- renderUI({
    lapply(vars(), function(v) {
      actionButton(paste0("var_", v), v)
    })
  })

  # Handle variable button clicks
  observe({
    for (v in vars()) {
      local({
        var <- v
        observeEvent(input[[paste0("var_", var)]], {
          updateTextInput(session, "expr",
                          value = paste0(input$expr, var))
        }, ignoreInit = TRUE)
      })
    }
  })

  # Connectives
  observeEvent(input$not, {
    updateTextInput(session, "expr", value = paste0(input$expr, "¬"))
  })
```

```
observeEvent(input$and, {
  updateTextInput(session, "expr", value = paste0(input$expr, "   "))
})

observeEvent(input$or, {
  updateTextInput(session, "expr", value = paste0(input$expr, "   "))
})

observeEvent(input$imp, {
  updateTextInput(session, "expr", value = paste0(input$expr, " → "))
})

observeEvent(input$iff, {
  updateTextInput(session, "expr", value = paste0(input$expr, "   "))
})

# Brackets (NEW)
observeEvent(input$lpar, {
  updateTextInput(session, "expr", value = paste0(input$expr, "("))
})

observeEvent(input$rpar, {
  updateTextInput(session, "expr", value = paste0(input$expr, ")"))
})

observeEvent(input$clear, {
  updateTextInput(session, "expr", value = "")
})

# Render table
output$table <- renderUI({

  req(vars())

  df <- generate_truth_table(vars())

  if (nchar(input$expr) > 0) {
    df$result <- evaluate_expr(input$expr, df)
```

```
    }

    format_val <- function(x) {
      if (isTRUE(x)) return('<span class="T">T</span>')
      if (identical(x, FALSE)) return('<span class="F">F</span>')
      return("")
    }

    html <- "<table>"

    # Header
    html <- paste0(html, "<thead><tr>",
                   paste0("<th>", c(vars(), "Result"), "</th>", collapse = ""),
                   "</tr></thead>")

    # Body
    html <- paste0(html, "<tbody>")

    for (i in 1:nrow(df)) {
      html <- paste0(html, "<tr>")

      for (v in vars()) {
        html <- paste0(html, "<td>", format_val(df[[v]][i]), "</td>")
      }

      if ("result" %in% colnames(df)) {
        html <- paste0(html, "<td>", format_val(df$result[i]), "</td>")
      }

      html <- paste0(html, "</tr>")
    }

    html <- paste0(html, "</tbody></table>")

    HTML(html)
  })
}
```

```
shinyApp(ui, server)
```

Tautologies, contradictions, and equivalence

While using the builder, you might have noticed that some statements are always true or always false, no matter the truth value of their components. These statements have a special name.

> **i** Definition of tautology
>
> A **tautology** is a proposition that is always true.

One example of a tautology might be $P \vee (\neg P)$ (check using the truth table builder!). This could be like saying '$x > 0 \vee x \leq 0$' which is always true; any real number always satisfies one of these statements.

> **i** Definition of contradiction
>
> A **contradiction** is a proposition that is always false.

A similar example of a contradiction might be $P \wedge (\neg P)$. This could be like saying '$x > 0 \wedge x \leq 0$,' which is never true; no real number satisfies both of these statements.

Another important idea is equivalence. This can be incredibly useful when working on mathematical proofs.

> **i** Definition of contrapositive
>
> You can say that two statements are logically **equivalent** if they agree on all truth values.

One very important equivalence is the **contrapositive.** Again, this can be very useful in mathematical proofs, you may find that sometimes it is easier to prove the **contrapositive** of an implication over the original implication. The **contrapositive** is logically **equivalent** to the original implication, so your proof will hold for the original implication!

> **i** Definition of contrapositive
>
> The **contrapositive** of $P \to Q$ is $(\neg Q) \to (\neg P)$.
> The **contrapositive** reverses an implication and negates both propositions.

Example 9

Let's return to the statement in example 5 and consider its contrapositive.
"$x > 5 \to y < 5$."
This is $P \to Q$ with P standing for "$x > 5$" and Q standing for "$y < 5$."
$\neg P$ would then mean "$x \leq 5$" and $\neg Q$ would be "$y \geq 5$."
So, $(\neg Q) \to (\neg P)$ would then mean "$y \geq 5 \to x \leq 5$." You can notice the equivalence to the original statement here. Let's verify this using truth tables.
The original statement was $P \to Q$, so its truth table would be:

P	Q	$P \to Q$
T	T	T
T	F	F
F	T	T
F	F	T

Now, let's look at the contrapositive, $(\neg Q) \to (\neg P)$.

P	Q	$\neg Q$	$\neg P$	$(\neg Q) \to (\neg P)$
T	T	F	F	T
T	F	T	F	F
F	T	F	T	T
F	F	T	T	T

So they do share a common truth table!

Quick check problems

1. You are given three statements below. Decide if they are propositions.

(a) $x = 5$.

(b) $y \geq 0$.

(c) $5 - 4 = 0$.

2. Which is the proper logical structure for the following statement: "If x is even, then $y = 10$."

(a) $P \vee Q$

(b) $P \to Q$

(c) $P \leftrightarrow Q$

3. How many rows does a truth table for 3 propositions have?

4. Is $P \to Q$ equivalent to $(\neg P) \vee Q$?

Further reading

For more questions on the subject, please go to Questions: Introduction to logic.

Version history

v1.0: initial version created 03/26 by Jessica Taberner as part of a University of St Andrews VIP project.

This work is licensed under CC BY-NC-SA 4.0.